

Testing a Preemptive RTOS on the Raspberry Pi Pico

Hadi

3302ENG Computer Systems

RTOS Laboratory Project

October 2026

Abstract—This report is about a small real-time operating system on a Raspberry Pi Pico. Several tasks share one processor. They show their work on an eight-digit seven-segment display and on a serial shell. The scheduler is preemptive round robin with three priority levels. A high task runs for 1 second, a medium task for 0.5 seconds, and a low task for 0.2 seconds. Those times are long enough to tell apart by eye. The shell is medium priority, so a high task started from it cuts in and a low task waits. We checked the board by watching the display and the shell, and by leaving it running for at least five minutes. The report also explains a fault in `remove_Task`. After the start-up task removed itself, the display froze on HELLO and the shell never started. The running task record had been pointed at the shell too early, so the shell's saved start state was overwritten.

Index Terms—RTOS, preemptive scheduling, round robin, Raspberry Pi Pico, testing

I. Introduction

The laboratory asks for a small operating system on a Raspberry Pi Pico. One processor has to share its time across several tasks. Each task is a function that loops and does not return. The tasks use the on-board LED, an eight-digit seven-segment display, and a USB serial shell.

The display board is wired to the Pico over SPI. Power and ground use the Pico supply pins. Chip select, clock, and data use pins 22, 24, and 25 [1]. A supplied hardware layer, running on the second core, copies an eight-byte buffer onto the glass. Our tasks only write that buffer. Byte 0 is the left digit.

The first scheduler gave every task the same short turn. This is round robin [2]. The second part of the laboratory asks for preemption from the scheduler, not from a new outside interrupt [1]. A more important new task should be able to start at once. The save and load of the processor registers are supplied. Our code chooses which task runs next.

This report explains why the scheduler is built this way, how we checked it, and what went wrong when the start-up task removed itself.

II. Why the Scheduler Works This Way

A. Preemption

The system tick comes from `SysTick`. The Pico clock is 125 MHz [3]. `configure_SysTick(1)` sets the reload value to $1 \times 125 - 1$, so the tick is 1 microsecond. The tick

handler counts a global `Ticks` value and also counts down the running task.

In the first version, every task gave up the processor after the same count. That count was 100 ticks, which is 100 microseconds. The scheduler then stepped to the next task in a linked list. At the end of the list it went back to the start. No new task could cut in early.

A more important task should not have to wait out someone else's whole turn. We kept the same tick, and we gave each task its own limit. The task record holds quantum and `ticks_left`. Each tick subtracts one from `ticks_left`. At zero, the handler refills the limit and asks for a context switch. The scheduler still walks the list in order for that kind of switch.

The cut-in is separate, and it happens when a task is added. If the new priority number is higher than the running task, `add_Task` stores the new task in `PreemptTarget` and asks for a switch. On that switch the scheduler runs `PreemptTarget` first and then clears it. A new task with the same priority, or a lower one, is only linked onto the list. It waits.

The shell is started at medium priority. A high task typed at the shell cuts in. A low task typed at the shell waits. Tasks still take turns after that. Priority sets the length of a turn, and it lets a higher new task start immediately. It does not keep the processor on the high task forever.

B. Why Three Levels

The laboratory leaves the meaning of the priority number to us [1]. We used three levels. A normal operating system groups work into a few bands, such as high, medium, and low. We used the numbers 3, 2, and 1 for those bands. Table I lists them.

TABLE I
The three priority levels

Level	Number	Time on each turn
High	3	1.0 s
Medium	2	0.5 s
Low	1	0.2 s

Another method is to let the priority number set the time. Priority 4 would run for four units, and priority 9

would run for nine units. Every different number would be its own speed. That is harder to explain, and it is harder to recognise on a small display. It also fits this program badly. `main` starts `initd` with priority 99. If the time followed the number, that one task would hold the processor for a very long turn.

With three fixed bands, the user tasks we care about are only low, medium, and high. Any other number is given the low time. Three bands look like a real system. A time that grows with the raw number does not.

C. Why the Shell Is Medium

The shell is started at priority 2. That is the medium band, so its turn is 0.5 seconds. It has to stay on that band for the preemptive part to work.

New tasks are created from the shell. When Enter is pressed, the running task is `cmdShell`. A new task cuts in only when its priority number is higher than the task that is running. The shell's number is therefore the line the new task has to cross.

The shell used to be priority 99, the same number `main` uses for `initd`. A high task is priority 3. Three is not higher than 99, so `add_Task` would only link the new task onto the list. The same thing happens if the shell itself is high. Three is not higher than three. In both cases `PreemptTarget` stays empty. Every task typed at the prompt, including a high one, waits for the shell's turn to end. During normal use the cut-in path never runs, because the shell is the only place tasks are added.

A low shell has the opposite problem. Medium and high are both higher than 1, so both cut in. Typing at the prompt can no longer show a task that waits. The shell would also take the short 0.2 second turn, so a high display task would keep the processor for a full second between chances to read the next key.

Medium is the band that leaves both results in place. A high task is above the shell, so its digit starts at once, while the shell still has time left. A low task is below the shell, so it waits until the 0.5 second turn ends. Another medium task is equal to the shell, so it does not cut in. It joins the list and shares later turns. The prompt stays in the middle: long enough to type a line, and short enough that the display tasks still get time.

This also keeps boot in the right order. `initd` runs at priority 99. The shell is 2, so adding the shell does not cut `initd` off. `initd` can still remove itself and wait for the next tick. That is the path in Section IV. If the shell were above 99, the switch would jump to the shell before `remove_Task` ran.

D. Why 1 Second, 0.5 Seconds, and 0.2 Seconds

The three limits are 1000000, 500000, and 200000 ticks. At 1 microsecond per tick, that is 1 second, 0.5 seconds, and 0.2 seconds.

We first used 100, 50, and 20 ticks. The ratio is the same, but those turns last 100 microseconds, 50 microseconds,

and 20 microseconds. The display tasks step a digit every 0.25 seconds or slower. A turn that short is over before the eye can see who had the processor. The glass looks as if every task is moving at once, so the levels cannot be told apart.

We kept the same ratio and stretched it until a person can watch it. One second, half a second, and 0.2 seconds are far enough apart. A high task holds a digit for about a second. A low task gets a short burst of about 0.2 seconds. These times are for the test. A finished product would use much shorter turns. Here the goal was to see the schedule on the screen.

III. How We Tested

A. What We Watched

We watched the USB serial shell and the seven-segment display. The shell prints shell `#` and reads a line when Enter is pressed. The commands we used are listed below.

- `pt` prints each task id, name, and priority.
- `rt` followed by an id removes that task.
- `sd` clears the display and stops the system.
- `count 0 3` starts the counter at digit 0 with high priority.
- `flash 2 1` flashes digit 2 with low priority.

The first number after the name is the task argument. The second number is the priority. `blink` takes a flash count. `count` takes the first digit of a pair. `flash`, `hexer`, and `splat` take one digit from 0 to 7.

`HELLO` uses the left five digits. `count` uses two digits. `flash`, `hexer`, and `splat` each use one. `blink` uses the LED on the Pico, on for 2 seconds and off for 2 seconds. Those delays only move forward while that task has the processor.

B. Start-up and Each Task

After reset we waited until the USB serial link was up. The Pico prints Starting RTOS Kernel, then Starting HAL. The left of the display shows `HELLO`. The shell then prints shell `#`. The command `pt` lists `cmdShell` only. `initd` has already removed itself.

We started each task from the shell and checked it against the laboratory script [1].

- `blink` flashes the LED the requested number of times, then leaves it off.
- `count` shows 00 to 99 and repeats. It steps after 0.5 seconds of its own running time.
- `flash` blinks the decimal point, on for 0.25 seconds and off for 0.25 seconds.
- `hexer` walks one digit from F down to 0 and back up to F, one step per second.
- `splat` turns segments on one by one, then off in reverse, one step every 0.25 seconds.

C. The Five Minute Run

A short look can miss a fault that shows up later. After the shell was up we started several display tasks at once, on different digits, and left the Pico running for at least five minutes.

We watched three things. The digits must keep changing. The pattern must not freeze on HELLO or on one number. The shell must still answer. We pressed Enter, ran pt, and checked that the same tasks were still listed. At the end of the five minutes the digits were still moving and the shell still answered. That was our check that the task list, the tick, and the display path stay alive together.

D. Telling the Three Times Apart

We ran the same kind of display task at different priorities so the time difference was easy to see.

A high task and a low task were placed on different digits. The high digit keeps updating for about a second at a time. The low digit updates in a shorter burst, about 0.2 seconds, and then waits while the other tasks take their turns. A medium task sits between those two. The steps inside count, flash, hexer, and splat are themselves a large fraction of a second, so the pause between bursts is visible.

We also checked the cut-in from the shell. The shell is medium. Starting a high task makes that task's digit move at once, while the shell still has time left in its turn. Starting a low task does not cut the shell off. The low digit starts to move after the current turn ends. That is the difference between cutting in now and waiting in the list.

IV. The Fault When initd Removed Itself

A. What We Saw

The laboratory says the first task, initd, must start the hardware layer, write HELLO, start the shell, and then remove itself [1]. We added that last step:

```
add_Task(&cmdShell, 0,
"cmdShell", PRIO_MED);
my_id = CurrentTCB->id;
remove_Task(my_id);
while (1) {}
```

The display showed HELLO and then froze. The serial port never printed shell #.

We commented out the two lines that read the current id and call remove_Task. The shell came back. count and flash could then draw over HELLO. HELLO was fine. The shell task was fine. Those two lines in initd are what the laboratory asks for. The fault was inside remove_Task, and it showed up when a task removed itself.

B. Why It Happened

A context switch always does the same four steps. The tick, or a cut-in request, asks for a switch. The supplied code saves the live registers onto the stack of whoever CurrentTCB points at. It then calls scheduler, which sets

CurrentTCB to the next task. It then loads that task's registers.

remove_Task is not a switch. It only edits the linked list. The processor stays inside the caller until the next tick. Each task's stack lives inside its own task record. CurrentTCB has to keep naming the task that is really running until the save has finished. If it names another task first, the save writes the live registers over that other task's stack.

The old function did unlink the node, and it did not free the running record. After the unlink it did one extra thing. If the removed task was the running task, it set CurrentTCB to the next task. The idea was to keep the scheduler off a dead node. That fights the switch, because the switch saves first and calls the scheduler second.

Just before the call, the list was initd, then cmdShell, and initd was running. initd is the head of the list, so the unlink leaves cmdShell as the only task in the list. CurrentTCB still pointed at initd, which was right. The extra lines then pointed CurrentTCB at cmdShell while the processor was still inside initd.

On the next tick the save wrote initd's registers over the shell's new stack. That stack held the dummy frame from add_Task, with the program counter set to cmdShell. The frame was destroyed. The scheduler loaded the damaged stack. The shell never started. The hardware layer on the other core kept copying the display buffer, so HELLO stayed on the glass.

The recovered stock remove_Task has a different fault on this same path. It never moves CurrentTCB, which is the right idea, but it always calls free on the task record. The stack is inside that record. Freeing the running task frees the memory the processor is still using. The next tick then saves onto memory the heap may already have reused. The shell can fail to start, or the board can hang, depending on what the heap does next.

The stock code was safe for one case only. The shell command rt removes some other task. The processor is not running on that task's stack, so free is safe. The template initd never removed itself. It looped forever. The laboratory does require the self-remove, so the stock function cannot be used as it stands. A missing id in the stock function also returns -1. The script asks for 0 [1].

C. The Fix and the Retest

The fix keeps the safe part of each version. It unlinks the id. It does not change CurrentTCB. It calls free only when the removed task is not the running task. The running task is left allocated on purpose. That one record stays until reset. rt of any other task still frees its record. A missing id returns 0.

The shell is medium, and initd was started with priority 99, so the shell does not cut in by preemption. initd has to leave the list and wait in while (1) for the tick. Returning from initd would be wrong as well. The dummy link register is empty. The task has to spin until the switch.

After the fix, the list is cmdShell only, and CurrentTCB still points at initd. The save uses initd’s stack, which was not freed. The scheduler follows initd’s old next pointer to the shell. The shell’s dummy frame is still intact, so the shell starts. pt does not list initd, because initd is no longer on the list.

Table II is the retest. We then left the display tasks running for at least five minutes, as in Section III-C. The shell stayed up and the digits kept moving.

TABLE II
Checks after the remove_Task fix

Check	What we saw
Boot display	HELLO on the left five digits
Serial	Starting HAL, then shell #
pt	cmdShell only
New tasks	count and flash draw over HELLO
Long run	Still running after five minutes

V. Conclusion

The Pico runs a preemptive round-robin scheduler with three priority levels. High, medium, and low get 1 second, 0.5 seconds, and 0.2 seconds. We used three bands because that is closer to a normal operating system than letting the run time follow the priority number. We used these three times so the difference is visible on the display. The shell is medium on purpose. Tasks are added from the shell, and a new task cuts in only when it is higher than the shell. High can cut in. Low has to wait. The test was to watch the glass and the shell, to start the tasks at low and high priority, and to leave the board on for at least five minutes.

A fault in remove_Task had pointed CurrentTCB at the shell while initd was still running. The next save overwrote the shell’s start frame, HELLO froze, and the shell never printed. Leaving CurrentTCB alone, and not freeing the running task, fixes that path. After the fix, HELLO appears, the shell starts, initd is gone from the list, and the display tasks keep running through the five minute check.

Acknowledgment

The context switch, the display hardware layer, and the helpers readln and delay came with the laboratory template. The scheduler, the task list, the tick handler, the shell, and the five tasks are our code.

References

- [1] 3302ENG Computer Systems, “Project 2: RTOS project,” laboratory script, Module 2, 2026.
- [2] 3302ENG Computer Systems, “(3A) RTOS implementation,” lecture notes, 2026.
- [3] Raspberry Pi Ltd., RP2040 Datasheet, system clock and SysTick timer.